# 01 · Product — Aiko, the agent on your laptop

## 01 · Product — Aiko, the agent on your laptop

> *Confidential DD dossier. Status tags:* ***BUILT+PROVEN / DESIGNED / RESEARCH-DIRECTION*** *(see* `00-README-DUE-DILIGENCE.md`*).* ***Value/economics layer:*** *how the network funds and rewards itself is the* ***reward-not-slash cooperative commons*** *— usage fees -> USDC to used + verified maintainers, nothing seized —* ***BUILT+PROVEN*** *on* `arc-lean`*. Full model:* `ECONOMICS-cooperative-commons.md` *(and* `03-ONCHAIN-*`*,* `04-PROOF-*`*).*

### The product in one line

**If you can do it on your laptop, ask Aiko.** Aiko is a local agent that does the things you'd do — reading your messages, understanding your context, driving your applications — so you don't have to.

### Who it's for

The core customer is **anyone who wants a personal agent that truly understands their context** — the existing user of ChatGPT / Claude / Claude Code who has tried an open agentic tool ("open-*") and found it unusable, unreliable, or unsafe on real work. Aiko is a **better open agent for computer use**: same ambition, but reliable enough and private enough to actually run on your machine, over your real data.

### What it feels like (the use cases)

- **Inbox triage in your voice.** Coworkers are bombing you on Telegram. Aiko reads the thread, tells you *what actually applies to you*, and drafts a reply *in your voice* — grounded in how you've replied before.
- **Real-world tasks, end to end.** It's your mum's birthday. "Find a restaurant nearby and book a table for six at 6:30." Aiko finds it, checks availability, and books it — a multi-step task across apps and the web.
- **Anything on the laptop.** The product surface is not a chat box; it is *your computer*. Aiko calls the right binaries and drives the right apps at the right time — less like a chatbot, more like reflexes: a muscle that contracts and fires the correct action for the moment.

### Why it beats a raw frontier LLM for computer use

A frontier LLM driving a computer is a brilliant generalist with no memory of what worked here before and no structure forcing it to be rigorous. Aiko wraps the model in a **harness that forces first-principles decomposition**: every task is broken into atomic sub-problems, and the model is made to reason mechanically — *which layer owns this step, where does the problem actually live, what fixes it* — rather than free-associating. The harness is the substrate; the model is one component inside it.

**The proof point (BUILT+PROVEN, demonstrated locally):** the harness — running Opus as the reasoner inside it — **rebuilt a Wine compatibility layer to launch and run Overwatch on a Mac in effectively one shot.** Not *playing* the game — getting a competitive Windows title to launch on macOS from a standing start. Most agents cannot do this; you cannot simply tell a general agent "make Overwatch run on my Mac" and have it work. It worked because the harness forced the model to solve the routing/travel problem one layer at a time, from first principles, and to locate and fix the actual failure — not because the base model got luckier. (See `04-PROOF-traction.md`.)

### The two halves: harness + a small grounded model

Aiko has two parts, and the distinction matters for diligence:

- **The harness is the Energy-Based Model.** It is a *recommendation/routing* system that learns which action to fire next from the collective history of what resolved before (see `02-ARCHITECTURE`). This is the durable, proprietary asset — **BUILT+PROVEN** as a router.
- **A small, non-parametric local model, grounded by the harness.** Separately, a compact instruction-following/reasoning model is being trained to be as *small* as possible so it runs locally. It "doesn't know much" on its own — but because it is **grounded** by the harness (which can feed it live, retrieved

knowledge about how the world and the machine work), it punches far above its size: smarter than the average model of the same footprint. Grounding, not parameter count, is where the capability comes from. **Status: the small model is DESIGNED / in-training; the grounding mechanism (retrieval + energy routing) is BUILT+PROVEN.**

The headline pitch — *"twice the reliability, half the cost"* — is the **design target** of this combination (a small grounded model routed by a cheap energy harness). It is not yet an independently benchmarked result; see the honest wall in `05-RISK-honest-wall.md`.

## What "learn as it works" means

Aiko learns **on-fire**: every time it declares an intent, satisfies it, or fails, that verdict folds into the harness's prior immediately — no training run, no labels, no pipeline. Satisfied = good, died = retry. The system's own experience *is* the training signal, and it is realtime: a new fact indexed or a new verdict fired sharpens the next decision now, not after the next retrain. **BUILT+PROVEN** (train-on-fire fold; energy-ranked resolution).

## Product status, plainly

Aiko is **used by the founder daily but not yet publicly released.** Two reasons, both honest: (1) the energy-based design is power-hungry and is being contained so it's safe and cheap for others to run; (2) the on-chain layer (which is how the weights become shareable and the storage cost is offloaded) is being hardened. The capability is real and demonstrated; the packaging is in progress.